

SKILL : WEEK 5

SESSION 9 :

Task -1 : To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main()
{
    char cwd[100], old[100];

    getcwd(cwd, sizeof(cwd));
    strcpy(old, cwd);

    printf("Current: %s\n", cwd);

    if (chdir("/tmp") == 0)
        printf("Changed to: /tmp\n");
    else
        perror("chdir");

    getcwd(cwd, sizeof(cwd));
    printf("Current: %s\n", cwd);

    if (chdir(old) == 0)
        printf("Returned to previous directory: %s\n", old);
    else
        perror("chdir");

    return 0;
}
```

OUTPUT:

```
ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Skill/Week5
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week5$ nano navigation.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week5$ nano navigation.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week5$ gcc navigation.c -o navigation
./navigation
Current: /mnt/d/OSSP_Programs/Skill/Week5
Changed to: /tmp
Current: /tmp
Returned to previous directory: /mnt/d/OSSP_Programs/Skill/Week5
```

OBSERVATION:

The program successfully demonstrated directory navigation and working directory management using the `getcwd()` and `chdir()` system calls. Initially, the current working directory was obtained and stored as the previous directory. The program then changed the working directory to `/tmp` and displayed the updated working directory to verify that the change was successful. The original directory was maintained in a separate variable and was later used to return to the previous working directory. Error handling using `perror()` was also included to report any failure while changing directories. The experiment demonstrated how a process can change and track its working directory while handling invalid or unsuccessful navigation operations. Thus, directory changing, path validation, working directory updates, previous-directory maintenance, and error handling were successfully demonstrated.

Task-2:

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <string.h>
#include <unistd.h>

void pwd()
{
    char dir[100];
    getcwd(dir, sizeof(dir));
    printf("Current directory: %s\n", dir);
}

void cd(char *path)
{
    if (chdir(path) != 0)
        perror("cd");
}

void exit_shell()
{
    printf("Shell terminated.\n");
}

int main()
{
    char command[100];

    // Dispatch table
    char *commands[] = {"pwd", "cd", "exit"};

    printf("Mini Shell\n");

    while (1)
    {
        printf("shell> ");
        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "pwd") == 0)
        {
            pwd();
        }
        else if (strncmp(command, "cd ", 3) == 0)
        {
            cd(command + 3);
        }
        else if (strcmp(command, "exit") == 0)
        {
            exit_shell();
            break;
        }
        else
        {
            printf("Invalid command.\n");
        }
    }

    return 0;
}
```

OUTPUT:

```
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week5$ nano builtins.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week5$ gcc builtins.c -o builtins
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week5$ ./builtins
Mini Shell
shell> pwd
Current directory: /mnt/d/OSSP_Programs/Skill/Week5
shell> cd /tmp
shell> pwd
Current directory: /tmp
shell> hello
Invalid command.
shell> exit
Shell terminated.
```

OBSERVATION:

The program successfully identified and executed the built-in commands `pwd`, `cd`, and `exit` using a dispatch mechanism. The built-in commands were stored in a dispatch table and executed directly within the current process without creating a separate child process. The `pwd` command displayed the current working directory, while the `cd` command successfully changed the working directory and maintained the updated state for subsequent commands. Invalid commands were detected and an appropriate error message was displayed. The `exit` command terminated the shell program successfully. Thus, built-in command identification, dispatch logic, in-process execution, state maintenance, invalid-command handling, and command testing were successfully demonstrated.

SESSION 10:

Task -1 :To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <stdlib.h>

int main()
{
    char cwd[100];
    char command[20];
    FILE *file;

    while (1)
    {
        printf("shell> ");
        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "pwd") == 0)
        {
            if (getcwd(cwd, sizeof(cwd)) != NULL)
                printf("Current Directory: %s\n", cwd);
            else
                perror("getcwd");
        }
        else if (strcmp(command, "exit") == 0)
        {
            if (getcwd(cwd, sizeof(cwd)) != NULL)
            {
                file = fopen("state.txt", "w");

                if (file != NULL)
                {
                    fprintf(file, "%s", cwd);
                    fclose(file);
                    printf("State saved.\n");
                }

                printf("Cleaning up resources...\n");
                printf("Exiting shell.\n");
                break;
            }
        }
        else
        {
            printf("Invalid command.\n");
        }
    }

    return 0;
}
```

OUTPUT:

```
ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Skill/Week5
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week5$ nano shell_state.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week5$ gcc shell_state.c -o shell_state
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week5$ ./shell_state
shell> pwd
Current Directory: /mnt/d/OSSP_Programs/Skill/Week5
shell> hello
Invalid command.
shell> exit
State saved.
Cleaning up resources...
Exiting shell.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week5$ cat state.txt
/mnt/d/OSSP_Programs/Skill/Week5ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week5$
```

OBSERVATION:

The program successfully retrieved and displayed the current working directory using the `getcwd()` system call. The `exit` command was used to process the user's request to terminate the shell, and before exiting, the current directory was saved to a file as the program state. The allocated file resource was properly closed using `fclose()`, demonstrating resource cleanup. The saved state was then verified by displaying the contents of the state file after the program terminated. Thus, retrieving and displaying the current directory, processing exit requests, saving state, cleaning up resources, and verifying the correct behavior were successfully demonstrated.

Task - 2:

To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/wait.h>

int valid_name(char *name)
{
    if (!isalpha(name[0]) && name[0] != '_')
        return 0;

    for (int i = 1; name[i] != '\0'; i++)
    {
        if (!isalnum(name[i]) && name[i] != '_')
            return 0;
    }

    return 1;
}

int main()
{
    char command[100];

    while (1)
    {
        printf("shell> ");
        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        if (strcmp(command, "export ", 7) == 0)
        {
            char *data = command + 7;
            char *equal = strchr(data, '=');

            if (equal == NULL)
            {
                printf("Invalid export syntax.\n");
                continue;
            }

            *equal = '\0';

            char *name = data;
            char *value = equal + 1;

            if (!valid_name(name))
            {
                printf("Invalid variable name.\n");
                continue;
            }

            setenv(name, value, 1);

            printf("Exported: %s=%s\n", name, value);
        }
        else if (strcmp(command, "test") == 0)
        {
            pid_t pid = fork();

            if (pid == 0)
            {
                printf("Child process: TEST_VAR = %s\n",
                    getenv("TEST_VAR"));
                exit(0);
            }
            else
            {
                wait(NULL);
            }
        }
        else
        {
            printf("Invalid command.\n");
        }
    }

    return 0;
}
```

OUTPUT:

```
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week5$ nano export.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week5$ gcc export.c -o export
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week5$ ./export
shell> export TEST_VAR=Hello
Exported: TEST_VAR=Hello
shell> test
Child process: TEST_VAR = Hello
shell> export TEST_VAR=World
Exported: TEST_VAR=World
shell> test
Child process: TEST_VAR = World
shell> export 123ABC=hello
Invalid variable name.
shell> export TEST_VAR
Invalid export syntax.
shell> exit
```

OBSERVATION:

The program successfully implemented the export functionality by parsing the NAME=value syntax and updating environment variables using `setenv()`. Variable names were validated to ensure that invalid names were rejected, and existing environment variables were successfully updated with new values. The program also demonstrated that exported variables are inherited by child processes created using `fork()`, where the value was retrieved using `getenv()`. Invalid export syntax and invalid variable names were handled appropriately with error messages. Thus, parsing export commands, updating and validating environment variables, handling existing variables, passing variables to child processes, and testing the functionality were successfully demonstrated.